Universidade Federal de Sergipe
Centro de Ciências Exatas e Tecnologia
Departamento de Ciência da Computação

Disciplina: PROJETO E ANÁLISE DE ALGORITMOS (PROCC0083)
Período: 2026.2
Turma: T01
Prof(a): GLAUCO DE FIGUEIREDO CARNEIRO
Equipe: 03

Relatório da AV2

1. Identificação da equipe

Amintas Gomes de Moraes Neto
Ântones do Nascimento Lima
Flavio Emanuel de Oliveira Santos
Gabriel Gomes Attie
Ismael Teles Santos
João Fernandes Santos Filho

2. Tema, corpus e fonte

Tema: 7 — Documentação de APIs públicas

Corpus: descrição OpenAPI 3.0.3 da GitHub REST API, versão 1.1.4 da descrição (versão da API 2026-03-10), com 1224 operações. Ficha do corpus: https://github.com/Khozex/PAA_UFS_2026_2_Neto_Amintas_Lima_Antones_Attie_Gabriel_Santos_Flavio_Teles_Ismael_Fernandes_Joao/blob/main/artefatos/ficha-do-corpus.pdf

Fonte: arquivo descriptions/api.github.com/api.github.com.json do repositório https://github.com/github/rest-api-description, licença MIT. Acesso em: 24 de setembro de 2026.

3. URL do repositório, licença, commit/tag/release e data de acesso

Código-fonte: https://github.com/Khozex/PAA_UFS_2026_2_Neto_Amintas_Lima_Antones_Attie_Gabriel_Santos_Flavio_Teles_Ismael_Fernandes_Joao

Licença: MIT

Release: tag av1 (entrega da AV1); tag av2 (entrega da AV2, commit 48c4f01).

Data de acesso: 24/09/2026

4. Definição formal do problema

O desenvolvimento de aplicações modernas depende amplamente do consumo de APIs, tornando a documentação técnica um recurso fundamental para que desenvolvedores possam identificar e utilizar corretamente os serviços e operações disponibilizados por essas interfaces. Entretanto, documentações extensas de APIs públicas podem apresentar grande quantidade de informações, distribuídas entre

diferentes categorias, recursos, métodos HTTP, parâmetros e regras de utilização, dificultando a localização manual do *endpoint* adequado para atender a uma determinada necessidade expressa em linguagem natural.

Nesse contexto, este trabalho propõe investigar o problema de recuperação de informação em documentação de APIs públicas, tendo como corpus a documentação da GitHub REST API, na versão de 10 de março de 2026. O problema pode ser formalizado da seguinte maneira: dada uma consulta em linguagem natural realizada por um desenvolvedor, identificar e recuperar o *endpoint* da API que melhor corresponde à operação solicitada.

Por exemplo, consultas como “Como criar um repositório no GitHub via API?”, “Como criar uma issue em um repositório?” ou “Como criar uma nova branch?” exigem que o sistema seja capaz de relacionar a intenção descrita pelo usuário às informações técnicas presentes na documentação, retornando a operação correspondente. Dessa forma, o problema não consiste apenas em localizar termos semelhantes, mas em estabelecer uma relação entre a intenção da consulta e a funcionalidade efetivamente oferecida por determinado *endpoint*.

A escolha da GitHub REST API (descrição OpenAPI 3.0.3, versão 1.1.4) como corpus é justificada por sua abrangência, diversidade e estruturação formal. A especificação OpenAPI disponibilizada oficialmente pelo GitHub permite representar cada operação de forma estruturada, contendo informações como método HTTP, caminho, parâmetros, descrição e *operationId*. Essa estrutura possibilita estabelecer uma unidade documental bem definida — o *endpoint* — e, principalmente, um identificador único para cada operação. Consequentemente, torna-se possível construir um gabarito objetivo de relevância para avaliar os resultados dos métodos de recuperação empregados.

O corpus selecionado apresenta aproximadamente 1.224 operações, distribuídas em 47 categorias e 140 subcategorias, totalizando cerca de 75 mil palavras de descrição. Essa dimensão proporciona um cenário suficientemente diversificado para avaliar a capacidade de diferentes estratégias de recuperação em distinguir operações semanticamente próximas, especialmente quando diferentes *endpoints* pertencem ao mesmo recurso ou apresentam descrições com vocabulário semelhante.

Assim, a investigação busca avaliar em que medida técnicas de recuperação de informação são capazes de mapear consultas formuladas em linguagem natural para operações técnicas específicas de uma API, utilizando uma fonte estruturada e reproduzível. A existência de uma correspondência objetiva entre consulta, *endpoint* e *operationId* também permite realizar uma avaliação quantitativa dos métodos utilizados, contribuindo para a análise da eficácia das estratégias de indexação, segmentação e recuperação aplicadas ao corpus.

5. Representação dos dados e estratégia de chunking

5.1 Representação dos dados e estratégia de chunking

Não foi utilizada estratégia de chunking. Cada documento é a operação completa. Os diferentes tamanhos de corpus exigidos pelo experimento são obtidos por subconjuntos dos N primeiros documentos do docs.jsonl, o que varia N sem alterar a unidade de recuperação nem os pesos por campo.
No contexto da avaliação 2, optou-se pela trilha C (Grafos e conhecimento).

Na AV1, cada operação da API era um documento isolado e o ranking usava TF-IDF. Aqui o corpus também vira um grafo, e três algoritmos (BFS, DFS e Dijkstra) fazem duas coisas:

1. achar operações ligadas aos resultados da busca ;
2. dizer em que ordem chamar as operações, como, por exemplo, listar antes de apagar.

5.1.1 Grafo × heurística de busca por texto

	Busca por texto com TF-IDF	**Busca no grafo**
Responde	"quais operações falam disso?"	"o que está ligado a isso?"
Bom para	achar pelas palavras da pergunta	achar vizinhos que não repetem as palavras; ordenar chamadas
Limitação	trata cada operação sozinha	precisa de um ponto de partida vindo da busca por texto

Exemplo (considerando corpus de teste com 74 operações): para "delete a release", o TF-IDF devolve o DELETE e o GET do release. O grafo, a partir deles, acrescenta o PATCH, os assets e a lista GET/POST …/releases. E o plano de chamadas sai assim: 1. GET …/releases → 2. DELETE …/releases/{release_id}. Essa ordem não sai do ranking por texto.

5.2 Especificação do problema da busca em grafo

Problema 1: achar evidências conectadas

- **Entrada**: o grafo; a consulta; as operações que a busca por texto devolveu (as "sementes", top-k); o método (BFS ou Dijkstra); um limite de saltos H (BFS) ou de custo B (Dijkstra); e um máximo de resultados L.
- **Saída**: lista de operações, sem repetir as sementes, da mais próxima à mais distante.
- **Restrições**: pesos nunca negativos; por padrão não se usa a aresta DependsOn.
- **Objetivo**: trazer as operações mais próximas das sementes.
- **Corretude**:

    - No BFS, a lista tem todas as operações a até H arestas de alguma semente, ordenadas pelo número de arestas;
    - No Dijkstra, a lista tem todas as operações com custo mínimo até B a partir de alguma semente, ordenadas por esse custo, e o caminho mostrado tem exatamente esse custo;
    - Em caso de empate, vale a ordem alfabética do ID.

Problema 2: ordem das chamadas

- **Entrada**: o grafo e a operação-alvo.
- **Saída**: lista de operações.
- **Corretude**: a lista termina no alvo, tem só o alvo e o que ele exige (direta ou indiretamente), e cada operação aparece depois das que ela exige. Se as dependências formarem um ciclo, a resposta é um erro, porque não existe ordem possível.

5.3 Corpus, consultas e parâmetros

Item	Valor
Corpus	data/processed/docs.jsonl: 1224 operações da GitHub REST API v1.1.4
Representação	Um documento por operação (método + path); vértices e arestas como acima
Tamanho do grafo	V = R + O vértices (R recursos, O = 1224 operações); E = 2(R−1) + 2·O + D arestas (D dependências)
Consultas	As 10 do gabarito data/queries.json, mais consultas livres
Sementes	k = 5 no CLI, k = 3 na interface; -preset 2
Expansão	H = 3, B = 3, L = 10
Limite de contexto	Por quantidade de operações (k, L) e, na interface, por caracteres (-max_chars). O grafo não conta tokens; isso fica com a camada RAG

5.4 Baseline

1. **Busca por texto da AV1** (TF-IDF, -preset 2, top-k). A comparação é "só o top-k" contra "top-k + expansão pelo grafo".
2. **Expansão simples**: para cada semente, pegar as operações com o mesmo começo de path, sem grafo e sem pesos.
3. **Busca linear**: serve para conferir que o ranking está certo, e chega no mesmo resultado que a indexada. Não substitui a busca em grafo.

Para o Problema 2 não há linha de base na AV1, pois o TF-IDF não ordena chamadas. A referência é a regra "lista antes do item".

5.5 Casos de borda

Caso	O que acontece	Teste
Corpus vazio	Grafo vazio; as buscas devolvem lista vazia, sem erro	TestEmptyAndDisconnected
Consulta sem resultado	Sem sementes, nada a expandir; devolve vazio	TestExpand
Semente que não está no grafo	É ignorada	TestEmptyAndDisconnected
Empate	Desempata por ID (e por índice do vértice no Dijkstra); o resultado deve ser sempre o mesmo	TestBuildStructure
Documento duplicado (mesmo método + path)	Vira uma operação só	TestDuplicateDocumentCollapses
Limite excedido	Trunca a lista, mantendo os mais próximos	TestBFSDepthLimitAndDijkstraBudget, TestExpand
Grafo desconexo / sem caminho	Distância infinita; Path avisa que não há caminho	TestEmptyAndDisconnected
Peso negativo ou inválido	AddEdge recusa	TestAddEdgeRejectsInvalid

Ciclo de dependência	TopoOrder e Plan devolvem erro. No grafo montado pelo Build não acontece, só em grafos feitos de maneira arbitrária	TestDFSDetectsCycle
Operação sem pré-requisito	O plano tem só ela	TestPlanOrdersPrerequisitesFirst

6. Algoritmos, pseudocódigos e estruturas de dados

6.1 Estrutura em grafo do corpus

Vértices: unidade de informação no grafo que pode ser um recurso ou operação

- Recurso: um pedaço do path, como /repos/{owner}/{repo}/releases;
- Operação: método + path, como DELETE /repos/{owner}/{repo}/releases/{release_id}. Cada operação é um documento do corpus.

Arestas: Ligam dois vértices e têm direção e um peso.

Aresta	**De → Para**	**Peso**	**Significado**
Contains	recurso → recurso filho	1	/repos contém /repos/{owner}
Within	recurso filho → recurso pai	1	o contrário da operação anterior
Exposes	recurso → operação	0,5	o recurso tem essa operação
ExposedBy	operação → recurso	0,5	o contrário

DependsOn	operação → operação	1	precisa ser chamada antes (ex.: DELETE …/releases/{id} precisa de GET …/releases, que dá o {id})

- **Regra da DependsOn**: para cada operação, sobe-se pelos ancestrais do seu path; sempre que um ancestral termina num parâmetro de identificador gerado pela API (nome terminado em _id, como {release_id}), e o pai desse ancestral tem uma operação GET, cria-se uma aresta da operação para esse GET, que é a listagem de onde o identificador vem. Parâmetros escolhidos pelo usuário, como {owner}, {repo} ou {branch}, não geram dependência.
- **Peso**: é a "distância" entre dois itens. Quanto menor, mais perto. Duas operações do mesmo recurso ficam a custo 1 (0,5 + 0,5). Os valores 1 e 0,5 foram escolhidos trivialmente, e não aprendidos dos dados.
- **Direção**: as quatro primeiras arestas existem nos dois sentidos, então dá para andar nos dois. Só DependsOn tem um sentido que importa: sai da operação e vai para o que ela exige antes.
- **Caminho**: uma sequência de vértices ligados por arestas. Por exemplo DELETE …/releases/{id} → recurso …/releases/{id} → PATCH …/releases/{id} mostra que as duas operações mexem no mesmo recurso. O custo do caminho é a soma dos pesos.

Grafo: lista de vértices e, para cada vértice, a lista de arestas que saem dele (Adj). Cada aresta guarda destino, peso e tipo.

Filtro: função que diz quais arestas podem ser usadas (por exemplo, só DependsOn).

BFS: uma fila, e vetores depth (nº de arestas desde a semente) e parent (de onde veio).

DFS: uma pilha, e uma cor por vértice (não visto, na pilha, terminado).

Dijkstra: vetores Dist (menor custo até agora) e Prev (de onde veio), e uma fila de prioridade (heap) ordenada por custo.

BFS

Anda por "camadas": primeiro as sementes, depois os vizinhos delas, e assim por diante. Ignora pesos e minimiza o número de arestas.

```
BFS(grafo, sementes, H):
  depth[v] = -1 para todo v
  para cada semente s: depth[s] = 0; coloca s na fila
  enquanto a fila não está vazia:
    u = tira o primeiro da fila
    se depth[u] == H: continue          //não passa do limite
```

para cada aresta u → v permitida:

se depth[v] == -1: //ainda não visitado

depth[v] = depth[u] + 1; parent[v] = u; coloca v na fila

DFS

Vai o mais fundo que puder por um caminho e só volta quando não há mais para onde ir. Aqui ela é feita com pilha (sem recursão). Serve para achar ciclos e para ordenar dependências.

DFS(grafo, sementes):

para cada semente s ainda não vista:

empilha s (cor: "na pilha")

enquanto a pilha não está vazia:

u = topo

se u não tem mais arestas: marca u como terminado; anota u em "Post"; desempilha

senão, para a próxima aresta u → v:

v não visto: empilha v

v "na pilha": achou um ciclo

Plan(alvo) = "Post" da DFS que parte do alvo seguindo só DependsOn

Dijkstra

Sempre escolhe o vértice não fechado mais barato, "fecha" esse vértice e atualiza o custo dos vizinhos (o chamado *relaxamento*: se passar por aqui sai mais barato, troca).

Dijkstra(grafo, sementes, B):

Dist[v] = infinito para todo v

para cada semente s: Dist[s] = 0; coloca s no heap

enquanto o heap não está vazio:

u = tira o de menor custo

se u já está fechado: continue

fecha u

para cada aresta u → v permitida, com peso w:

novo = Dist[u] + w

se novo > B: continue (passou do orçamento)

se novo < Dist[v]: Dist[v] = novo; Prev[v] = u; coloca v no heap

Como foi verificado nos testes

- Dijkstra comparado com o algoritmo de Bellman-Ford (que calcula o mesmo resultado de outro jeito) em 300 grafos aleatórios, com pesos de 0 a 3,5.
- BFS e Dijkstra dão o mesmo resultado quando todos os pesos são 1 (200 grafos).
- Em um grafo onde os dois divergem, como 1 aresta de custo 10 contra 3 arestas de custo 1, a BFS escolhe a de menos arestas, o Dijkstra a de menor custo.
- A ordem do Plan respeita todas as arestas DependsOn do grafo da API.
- Montar o grafo duas vezes dá o mesmo grafo.

7. Justificativa de corretude

BFS

Prova de corretude: A fila sempre está em ordem de depth, e do primeiro ao último a diferença é no máximo 1. Dá para mostrar por indução: as sementes têm depth 0. Se um vértice está a d+1 arestas, o vizinho dele a d arestas sai da fila antes e o descobre com d+1. Ele não pode ter sido descoberto com um valor menor, senão existiria um caminho mais curto. Parar em H só deixa de expandir os vértices em H, o que não muda os que estão até H.

DFS

Prova de corretude

- Ciclo: voltar para um vértice que ainda está na pilha significa que há um caminho dele até u e uma aresta de u de volta a ele, ou seja, um ciclo. Se existe um ciclo, a DFS acaba passando por uma aresta assim, então ela não deixa ciclo escapar.
- Ordem: se u exige v e não há ciclo, v termina **antes** de u (ou já tinha terminado, ou é visitado dentro de u). Como Post lista os vértices na ordem em que terminam, os pré-requisitos vêm primeiro e o alvo por último. É isso que o Plan devolve.
- A DFS a partir do alvo visita só o que ele exige, direta ou indiretamente.

Dijkstra

Prova de corretude: Quando um vértice é fechado, o Dist dele já é o menor possível. Prova por contradição: suponha que u foi fechado com um custo maior que o ótimo. No caminho ótimo até u existe um primeiro vértice y ainda não fechado, e o anterior a ele já foi fechado e o relaxou, então Dist[y] está certo. Como nenhum peso é negativo, y custa no máximo o mesmo que u, então sairia do heap antes dele.

Contradição. Daí segue:

- os vértices são fechados em ordem crescente de custo;
- várias sementes funcionam como uma única semente ligada a todas por arestas de custo 0;
- o orçamento B não atrapalha: se um caminho custa até B, qualquer pedaço inicial dele também custa até B, então cortar o que passa de B não perde nenhum caminho dentro do limite.

Hipóteses de invalidade de corretude

É fundamental apresentar as hipóteses de contraposição dos argumentos de corretude anteriores: a explicação de *por que* cada algoritmo dá a resposta certa, como a fila ordenada da BFS, "quem termina primeiro" na DFS e "o vértice fechado já está no menor custo" no Dijkstra. Esses argumentos não valem sempre: cada um depende de condições específicas. Se a condição for quebrada, não há garantia de que o resultado obtido pela execução da implementação do algoritmo está correto.

Motivação inicial

As contraposições de corretude são importantes pois:

- Permitem a inferência da corretude de um resultado;
- Evita um erro silencioso: sem a lista de hipóteses, alguém poderia mudar o grafo, por exemplo, pôr um peso negativo, e continuar achando que o resultado é o menor caminho.
- Separa o que foi provado do que é suposição, que é o caso da expansão trazer o que é relevante.
- Mostra onde os testes precisam "olhar": cada hipótese é um caso de teste.

Hipóteses

Hipótese	Algoritmo	Se for violada
Pesos maiores ou iguais a 0	Dijkstra	O caminho pode não ser o de menor custo. Por isso AddEdge recusa pesos negativos
Sem ciclo em DependsOn	Plan e ordenação topológica	Não existe ordem válida; o código devolve erro com o ciclo

Grafo não muda durante a busca	DFS, BFS e Dijkstra	Os invariantes (fila ordenada, custos fechados) deixam de valer. Depois do Build o grafo só é lido, então é seguro usar em paralelo.
Pesos definidos	Dijkstra	O computador representa 1 e 0,5 sem erro. Com pesos como 0,1 pode haver erros de arredondamento que mudam um desempate.
A BFS só vale para "menos arestas"	BFS	Se os pesos forem diferentes, ela **não** dá o caminho mais barato; use Dijkstra.
As provas valem para as arestas que o filtro deixa passar	DFS, BFS e Dijkstra	Plan usa só DependsOn; a expansão usa as demais. Misturar muda o resultado.
"Perto" quer dizer "relevante"	Expansão	Sem garantia. Pode trazer operações próximas, mas inúteis com base no contexto de busca esperado.
DependsOn deduzida do path está certa	Plan	A regra só reconhece identificadores terminados em _id. Números como {issue_number} e {pull_number} são tratados como conhecidos pelo usuário e não geram dependência; se o usuário não souber o número, o plano não inclui a listagem. A dependência também falta quando o pai do identificador não tem GET.
Corpus real parecido com o de teste	DFS, BFS e Dijkstra	Paths muito diferentes podem mudar a estrutura e o desempenho. No corpus atual, o TestGraphOnRealCorpus confere, com as 1224 operações e com 612, que toda DependsOn aponta para um GET acima no path, que $E = 2(R-1) + 2 \cdot O + D$ e que não há ciclo.

8. Análise no modelo RAM

A análise a seguir considera o modelo RAM (*Random Access Machine*), no qual operações elementares sobre dados de tamanho limitado possuem custo constante. Assim, atribuições, comparações, operações aritméticas, leitura e escrita em posições de vetores, acesso a índices e atualização de contadores são consideradas operações de custo $\Theta(1)$.

Para as tabelas hash utilizadas pela implementação, como os mapas empregados para localizar vértices, documentos e operações, considera-se custo esperado $\Theta(1)$ para busca e inserção. Operações sobre heap e ordenações são analisadas separadamente, pois seu custo não é constante.

Serão utilizados os seguintes parâmetros:

$n = |V|$: número de vértices do grafo;

$m = |E|$: número de arestas;

$k = |S|$: número de sementes ou resultados utilizados como ponto inicial;

r: número de operações relacionadas produzidas durante uma expansão;

I: número total de inserções realizadas no heap do Dijkstra;

d: profundidade considerada na árvore de recursos ou no caminho de uma operação.

A implementação representa o grafo por listas de adjacência. Cada vértice possui uma lista contendo suas arestas de saída, e cada aresta armazena destino, peso e tipo. Dessa forma, percorrer os vizinhos de um vértice (v) custa Θ(deg(v)), enquanto percorrer todas as listas de adjacência custa:

$$\Theta(n + m)$$

Essa representação é adequada ao corpus utilizado, pois o grafo é esparso: no corpus completo foram construídos 2.214 vértices e 4.737 arestas.

8.1 Construção do grafo

A construção do grafo percorre a estrutura hierárquica derivada dos caminhos da API. Para cada recurso é criado um vértice e, quando existe um recurso pai, são adicionadas as arestas estruturais nos dois sentidos. Para cada operação associada ao recurso, também é criado um vértice e são adicionadas as arestas que relacionam recurso e operação.

Além dessas relações estruturais, a implementação cria as arestas DependsOn. Para cada operação, são percorridos os ancestrais de seu caminho em busca de parâmetros de identificadores gerados pela API, como {release_id}. Quando uma dependência válida é encontrada, é criada uma aresta para a operação GET correspondente.

Considerando (d) como a profundidade máxima do caminho analisado, essa etapa pode exigir, para cada operação, até (O(d)) acessos aos ancestrais. Assim, o custo dessa parte é:

$$O(Od)$$

onde (O) representa o número de operações do corpus.

Ao final da construção, as listas de adjacência são ordenadas para garantir comportamento determinístico. Se(deg(v)) é o grau de saída de um vértice, o custo total dessa ordenação é:

$$\sum_{v \in V} O(\deg(v) \log \deg(v))$$

No pior caso, esse custo pode ser limitado por:

$$O(m \log m)$$

Portanto, uma expressão geral para o custo da construção é:

$$O(n + m + Od + m \log m)$$

Como os caminhos da API possuem profundidade limitada e o grafo apresenta baixo grau médio, na prática esse custo permaneceu próximo de linear no tamanho do corpus. Nos experimentos, ao dobrar o corpus de 612 para 1.224 operações, o número de vértices e arestas praticamente dobrou e o tempo de construção passou de aproximadamente 440 μs para 970 μs.

A representação do grafo utiliza memória proporcional ao número de vértices e arestas:

$$O(n + m)$$

8.2 BFS

A BFS utiliza uma fila, o vetor depth, que armazena a distância em número de arestas desde as sementes, e o vetor parent, utilizado na reconstrução dos caminhos.

A fila é implementada utilizando um vetor e um índice que indica o próximo elemento a ser removido. Dessa forma, tanto a inserção quanto a remoção lógica da fila possuem custo constante:

$$\Theta(1)$$

A inicialização dos vetores percorre todos os (n) vértices:

$$\Theta(n)$$

Cada vértice alcançado é inserido na fila no máximo uma vez, pois o vetor depth impede uma segunda descoberta. Cada aresta permitida dos vértices efetivamente explorados também é examinada no máximo uma vez.

Assim, se (n') e (m') representam, respectivamente, o número de vértices e arestas efetivamente explorados até o limite (H), o custo da busca é:

$$\Theta(n + n' + m')$$

Como $(n' \leq n)$ e$(m' \leq m)$, no pior caso:

$$\Theta(n + m)$$

O espaço auxiliar utilizado por depth, parent e pela fila é:

$$\Theta(n)$$

O limite (H) não altera o pior caso assintótico, mas reduz o número de vértices e arestas efetivamente examinados nas execuções reais.

8.3 DFS e planejamento de chamadas

A DFS utilizada no projeto é iterativa. A implementação mantém uma pilha explícita, um vetor que indica o estado de cada vértice e uma estrutura que registra qual aresta deverá ser examinada em seguida.

Cada vértice alcançado é empilhado e removido um número constante de vezes. A estrutura que guarda a próxima aresta evita reiniciar o percurso da lista de adjacência quando a execução retorna a um vértice. Dessa maneira, cada aresta permitida é examinada no máximo uma vez em uma execução da DFS.

Para uma única execução:

$$\Theta(n + m)$$

no pior caso.

A memória auxiliar é composta principalmente pelos estados dos vértices, pela pilha, pelos predecessores e pela ordem de término, totalizando:

$$\Theta(n)$$

A operação de planejamento utiliza a DFS filtrando apenas arestas do tipo DependsOn. Para uma operação-alvo, seu custo depende apenas do subgrafo de dependências alcançável a partir dessa operação.

Entretanto, a função que produz planos para vários resultados executa uma DFS separada para cada uma das (k) operações recebidas. Assim, no pior caso, o custo do planejamento de todos os resultados é:

$$O(k(n + m))$$

Esse limite é pessimista. No corpus atual, o subgrafo formado por DependsOn é pequeno e esparso. Experimentalmente, a mediana observada foi de apenas cinco arestas examinadas no planejamento, tanto para o corpus de 612 quanto para o de 1.224 operações.

8.4 Dijkstra

O Dijkstra mantém os vetores de distância e predecessores, uma marcação dos vértices já fechados e uma fila de prioridade implementada por heap mínimo.

Inicializar as estruturas correspondentes aos vértices possui custo $\Theta(n)$.

Para cada vértice fechado, suas arestas permitidas são examinadas. O cálculo:

$$novo = Dist[u] + w$$

e a comparação com a distância já conhecida possuem custo $\Theta(1)$.

A implementação não realiza uma operação explícita de decrease-key. Quando uma distância é melhorada, uma nova entrada para o vértice é inserida no heap. Entradas antigas tornam-se obsoletas e são descartadas posteriormente quando removidas da fila de prioridade.

Se (I) representa o número total de inserções no heap, cada operação de inserção ou remoção custa:

$$O(\log I)$$

Como cada relaxamento bem-sucedido está associado à análise de uma aresta:

$$I \leq m + k$$

Portanto, uma forma que representa diretamente a implementação é:

$$O(n + m + I \log I)$$

e, substituindo o limite de (I):

$$O(n + m + (m + k) \log(m + k))$$

Considerando (k \leq n), pode-se expressar o pior caso de forma simplificada como:

$$O((n + m) \log n)$$

nas condições adotadas para o grafo utilizado.

Os vetores Dist, Prev e de vértices fechados ocupam (O(n)). Como podem existir múltiplas entradas no heap para o mesmo vértice, seu tamanho pode crescer proporcionalmente ao número de relaxamentos realizados. Assim, o espaço auxiliar no pior caso é:

$$O(n + m)$$

O limite de orçamento (B) utilizado pela implementação impede a exploração de caminhos cujo custo ultrapasse o valor definido. Assim como ocorre com (H) na BFS, esse limite não altera a complexidade assintótica de pior caso, mas reduz significativamente o trabalho realizado nos experimentos.

8.5 Custo da expansão de contexto

A operação de expansão não termina imediatamente após a execução da BFS ou do Dijkstra. Os vértices encontrados ainda são transformados em resultados a serem enviados à etapa de recuperação de contexto.

Para cada operação relacionada, a implementação reconstrói o caminho entre a semente e o vértice recuperado. Se o comprimento máximo desses caminhos for (p), a reconstrução de (r) resultados possui custo:

$$O(rp)$$

Na expansão por BFS, como a profundidade é limitada por (H):

$$p \leq H$$

e, portanto:

$$O(rH)$$

Após a construção dos resultados, as (r) operações relacionadas são ordenadas antes da aplicação do limite máximo de resultados. Essa ordenação possui custo:

$$O(r \log r)$$

Consequentemente, o custo completo de uma expansão utilizando BFS pode ser representado por:

$$O(n + m' + rH + r \log r)$$

onde (m') representa as arestas efetivamente examinadas.

Para a expansão com Dijkstra:

$$O(n + m' + I \log I + rp + r \log r)$$

É importante observar que o limite (L) de resultados é aplicado depois da busca e da ordenação. Assim, definir, por exemplo, (L=10) limita o contexto retornado, mas não implica que apenas dez vértices sejam processados pelo algoritmo.

8.6 Síntese da análise RAM

A análise mostra que BFS e DFS possuem custo linear em relação à região do grafo explorada, enquanto o Dijkstra possui custo adicional decorrente da fila de prioridade. A construção do grafo introduz um custo de pré-processamento, mas a estrutura pode ser reutilizada por todas as consultas seguintes.

Os limites (H) e (B) reduzem a região efetivamente explorada, enquanto (L) limita apenas a quantidade de resultados retornados. Dessa forma, embora as complexidades de pior caso permaneçam dependentes de (n) e (m), os experimentos mostram que a busca percorre apenas uma pequena vizinhança das sementes no corpus utilizado.

9. Melhor, pior e caso médio

9.1 BFS

Entrada do algoritmo:

- **G** = (**V**,**E**): grafo direcionado, com conjunto de vértices **V** e arestas **E**;
- **S**: conjunto de sementes, ou vértices iniciais;
- **H**: profundidade máxima permitida;
- **d(v)**: profundidade atribuída ao vértice **v**;
- **Q**: fila utilizada para explorar os vértices em largura.

Definiremos:

- n = |**V**|: número de vértices;
- m = |**E**|: número de arestas;
- k = |**S**|: número de sementes;
- **R_H**: conjunto de vértices alcançáveis a partir das sementes com distância mínima de, no máximo, **H**.

9.1.1 Melhor Caso:

O melhor caso ocorre quando o algoritmo executa o mínimo de trabalho possível, considerando o grafo e os parâmetros de entrada.

Por exemplo, isso acontece quando **H** = 0, onde todas as sementes são inseridas na fila, mas, ao retirar cada uma delas, o algoritmo encontra depth[u] == **H** e não percorre nenhuma aresta.

Nesse cenário, são realizadas as seguintes operações:

- Inicialização do vetor depth: **n** operações;
- Processamento das sementes: Ω(k)
- Remoção das sementes da fila e verificação da profundidade: Ω(k).

Portanto, o custo temporal é **Ω(n + k)**.

9.1.2 Caso Médio:

O caso médio depende da distribuição das entradas consideradas. Para calculá-lo rigorosamente, precisamos definir como os grafos, as sementes e o limite **H** são escolhidos.

Como somente os vértices com profundidade menor que **H** têm suas listas de adjacência percorridas, se os grafos considerados forem suficientemente conectados, a maioria dos vértices for alcançável e **H** permitir explorar boa parte do grafo, o custo médio poderá se aproximar de **Θ(n + m)**.

9.1.3 Pior Caso:

O pior caso ocorre quando o algoritmo precisa percorrer o maior número possível de vértices e arestas.

Considere um grafo no qual:

- Todos os vértices são alcançáveis a partir das sementes;
- O limite **H** é suficientemente grande para permitir a exploração de todo o grafo alcançável;
- As listas de adjacência de todos os vértices alcançados precisam ser percorridas integralmente.

Nesse cenário, temos:

- Inicialização do vetor depth: O(n);
- Processamento das sementes: O(k);
- Processamento dos vértices: O(n);
- Percurso das arestas: O(m).

Somando os custos, temos: **O(n + m + k)**

9.1.4 Complexidade Espacial:

O BFS utiliza “depth” como estrutura auxiliar de armazenamento. Esta variável armazena um valor para cada vértice do grafo, logo sua complexidade espacial é **O(n).**

9.2 DFS

Considere:

- **G** = (**V**,**E**): grafo direcionado, com conjunto de vértices **V** e arestas **E**;
- n = |**V**|: número de vértices;
- m = |**E**|: número de arestas;
- k = |**S**|: número de sementes;
- **R**: conjunto de vértices alcançados pela DFS a partir das sementes;
- **E_R**: conjunto de arestas examinadas durante a execução.

9.2.1 Melhor Caso:

O melhor caso ocorre quando a DFS realiza o mínimo de trabalho possível para a entrada fornecida.

Por exemplo, se não houver sementes, nenhum vértice será visitado. Se houver uma única semente sem arestas de saída, a DFS apenas a empilhará, marcará como terminada e a registrará em Post.

Se o algoritmo inicializar os estados de todos os vértices, essa inicialização custa Ω(n). O processamento das sementes custa, no máximo, Ω(k), e a exploração dos vértices alcançados custa Ω(|**R**| + |**E_R**|).

Como o melhor caso pode ocorrer com uma quantidade constante de sementes e sem arestas percorridas, temos **Ω(n)**.

9.2.2 Caso Médio:

O custo médio depende da distribuição dos grafos, da quantidade de sementes e da conectividade entre os vértices. Se, em média, a DFS alcançar uma grande parcela dos vértices e examinar uma grande parcela das arestas, seu custo médio poderá se aproximar de **Θ(n + m).**

9.2.3 Pior Caso:

O pior caso ocorre quando a DFS visita todos os vértices e examina todas as arestas do grafo.

Isso pode acontecer quando uma semente alcança todos os vértices ou quando as sementes, em conjunto, alcançam todo o grafo.

Nesse caso temos:

- Inicialização dos estados: O(n);
- Processamento dos vértices: O(n);
- Processamento das arestas: O(m).

Portanto, considerando $k \leq n$, temos **O(n + m)**.

9.2.4 Complexidade Espacial:

A DFS utiliza estruturas auxiliares para armazenar os estados dos vértices, a pilha e a lista Post.

No pior caso:

- Estados dos vértices: O(n);
- Pilha: O(n);
- Lista Post: O(n);
- Iteradores das listas de adjacência: O(n), se forem armazenados por vértice.

Portanto, a complexidade espacial é **O(n)**.

9.3 Dijkstra

Considere:

- **G** = (**V**,**E**): grafo direcionado, com conjunto de vértices **V** e arestas **E**;
- n = |**V**|: número de vértices;
- m = |**E**|: número de arestas;
- k = |**S**|: número de sementes;
- **B**: orçamento máximo permitido;
- **Q**: heap de prioridade mínima;
- **R_B**: conjunto de vértices fechados durante a execução;
- **E_B**: conjunto de arestas examinadas a partir dos vértices fechados.

9.3.1 Melhor Caso:

O melhor caso ocorre quando o algoritmo realiza a menor quantidade possível de trabalho para a entrada considerada.

Por exemplo, suponha que não existam sementes. O heap começa vazio e o laço principal não é executado. Entretanto, o algoritmo ainda inicializa o vetor de distâncias e, normalmente, o vetor de vértices fechados. A inicialização custa $\Omega(n)$.

Outro exemplo é quando $\mathbf{B} = 0$ e todas as arestas possuem pesos estritamente positivos. Nesse caso, as sementes podem ser processadas, mas nenhuma aresta permite atualizar a distância de um vértice para um valor dentro do orçamento. Se houver **k** sementes, o custo será $\Omega(n + k \log k)$.

Portanto, como a inicialização percorre todos os vértices, o melhor caso gera é **$\Omega(n)$.**

9.3.2 Caso Médio:

O caso médio depende da distribuição dos grafos, dos pesos das arestas, das sementes e do orçamento **B**. Se, em média, o algoritmo examinar uma grande parcela das arestas e realizar um número de atualizações proporcional a **m**, o custo poderá se aproximar de **$\Theta((n + m) \log n)$.**

9.3.3 Pior Caso:

Para o pior caso, temos que considerar que:

- Custo da inicialização: $O(n)$;
- Processamento do heap: $O(I \log I)$, sendo I o número total de inserções realizadas no heap, incluindo as sementes;
- Processamento das arestas: $O(m)$

Como cada aresta é examinada no máximo uma vez, o número de inserções adicionais é limitado por **m**.

Logo $I \leq m + k$.

Portanto, uma expressão para o pior caso é $O(n + m + (m + k) \log (m + k))$. Se $k \leq n$ e o grafo não possui arestas paralelas, então $m \leq n^2$.

Logo podemos simplificar para **$O((n + m) \log n)$**.

9.3.4 Complexidade Espacial

O algoritmo utiliza:

- Vetor Dist: $O(n)$;
- Vetor de vértices fechados: $O(n)$;
- Vetor Prev: $O(n)$;
- Heap: $O(m+k)$.

Assim, a complexidade espacial no pior caso é O(n + m + k). Se k ≤ n, então podemos simplificar para **O(n + m)**.

10. Recorrências

Nesta implementação, os algoritmos BFS, DFS e Dijkstra foram desenvolvidos de forma iterativa. A BFS utiliza uma fila, a DFS utiliza uma pilha explícita e o Dijkstra utiliza uma fila de prioridade baseada em heap. Portanto, não há recorrências associadas diretamente à execução desses três algoritmos.

A parte recursiva da implementação aparece principalmente durante a construção do grafo, quando a função responsável pelo percurso da árvore de recursos visita recursivamente os nós filhos.

10.1 Percurso recursivo da árvore de recursos

Durante a construção do grafo, cada recurso da árvore é processado uma única vez. Para cada recurso, a implementação:

- cria o vértice correspondente;
- adiciona as arestas que o relacionam ao recurso pai;
- percorre as operações associadas ao recurso;
- cria os vértices e as arestas dessas operações;
- chama recursivamente a mesma função para cada recurso filho.

Assim, o custo de processar um recurso corresponde ao custo de processar seus filhos mais o trabalho local realizado naquele recurso.

De forma simplificada, a recorrência pode ser expressa como:

$$T(v) = \sum T(filho) + \Theta(1 + o(v))$$

onde $o(v)$ representa a quantidade de operações associadas ao recurso v.

Como cada recurso e cada operação são visitados uma única vez, o custo total desse percurso é linear em relação ao número de recursos e operações. Dessa forma, considerando R recursos e O operações, o custo do percurso é:

$$\Theta(R + O)$$

Como esses elementos correspondem aos vértices principais construídos no grafo, pode-se também considerar esse custo linear em relação ao número de vértices.

10.2 Árvore degenerada

No caso em que cada recurso possui apenas um único filho, a árvore assume uma estrutura semelhante a uma lista encadeada.

Nesse cenário, cada chamada recursiva processa um recurso e chama a função para o próximo recurso. A recorrência pode ser representada por:

$$T(n) = T(n - 1) + \Theta(1)$$

A solução dessa recorrência é linear:

$$T(n) = \Theta(n)$$

Portanto, mesmo em uma árvore totalmente desbalanceada, o percurso continua visitando cada recurso apenas uma vez.

10.3 Árvore balanceada

Em uma árvore aproximadamente balanceada, cada recurso pode gerar várias chamadas recursivas para seus filhos.

Apesar da distribuição diferente das chamadas, o comportamento assintótico permanece linear, pois nenhuma parte da árvore é processada mais de uma vez. Cada recurso e cada operação pertencem a uma única subárvore e são visitados uma única vez.

Assim, o custo total continua sendo proporcional à quantidade de elementos processados, isto é, $\Theta(R + O)$.

10.4 Relação com o custo total da construção

A recorrência anterior descreve apenas o percurso recursivo da árvore de recursos.

A função completa de construção do grafo executa outras operações além desse percurso, como:

- criação das arestas `DependsOn`;
- busca por ancestrais dos caminhos da API;
- ordenação das listas de adjacência;
- ordenação de chaves utilizada para manter o resultado determinístico.

Portanto, o custo total da construção do grafo é superior ao custo do percurso recursivo isolado.

Conforme discutido na análise RAM, a construção também depende do número de arestas, da profundidade dos caminhos e do custo das ordenações realizadas.

10.5 BFS, DFS e Dijkstra

Não são apresentadas recorrências para BFS, DFS e Dijkstra porque a implementação atual dos três algoritmos é iterativa.

Na BFS, a repetição ocorre através do processamento da fila até que não existam mais vértices a explorar ou que o limite de profundidade seja atingido.

Na DFS, embora o algoritmo seja frequentemente apresentado de forma recursiva na literatura, a implementação do projeto utiliza uma pilha explícita. Portanto, o comportamento é analisado diretamente pelo número de vértices e arestas processados, e não por recorrência.

No Dijkstra, o processamento é controlado pela fila de prioridade. A cada iteração, o vértice de menor custo é retirado do heap e suas arestas são examinadas. Quando uma distância melhor é encontrada, uma nova entrada é inserida no heap.

Assim, para esses algoritmos, a análise por contagem de operações apresentada no modelo RAM e na seção de melhor, pior e caso médio é mais adequada do que uma formulação por recorrência.

11. Metodologia experimental

Foram feitos dois experimentos: um de custo (tempo, memória e trabalho do grafo) e um de qualidade (se o contexto vindo do grafo ajuda a LLM a responder).

11.1 Ambiente

Linux x86_64 (Fedora), AMD Ryzen 7 7800X3D (8 núcleos, 16 threads), 30,5 GiB de RAM e Go 1.26.5. As LLMs rodaram localmente no Ollama, dentro de um contêiner podman com ROCm, numa GPU AMD RX 9070 XT de 16 GB. O cabeçalho de cada arquivo de saída registra o hardware, o commit e o comando usado.

11.2 Experimento de custo do grafo

- Cenários: 4 configurações × 2 tamanhos de corpus = 8 cenários.
    - Configurações: só o top-k do TF-IDF (base); top-k + expansão por BFS; top-k + expansão por Dijkstra; top-k + plano de chamadas.
    - Tamanhos: N = 1224 operações (corpus completo) e N = 612 (os 612 primeiros documentos, opção -limit 612).
- Consultas: as 10 do gabarito data/queries.json, com 3 repetições cada, num total de 240 execuções.
- Parâmetros: k = 5, -preset 2, H = 3, B = 3, L = 10.
- Comando base: bin/search -query Q -k 5 -preset 2 [-limit 612] [-expand bfs|dijkstra -hops 3 -budget 3 | -plan].
- Métricas por execução: tempo de carga, de construção do índice, de construção do grafo, de consulta, de expansão e do plano; memória; tamanho do grafo (V, E); vértices visitados e arestas

examinadas; número de relacionadas e de pré-requisitos; posição da resposta certa; e se a resposta certa apareceu depois da expansão.
- Saídas: artefatos/benchmark_grafo.csv e benchmark_grafo.txt.

11.3 Experimento com a LLM

- Perguntas: 20 perguntas em português do gabarito data/queries_av2.json. Cada uma vem com a consulta em inglês, os operationIds considerados corretos (relevant) e os pré-requisitos (prerequisites): os passos que vêm antes da operação, cada um com as chamadas que servem para ele. Quatro dessas perguntas não têm resposta no corpus e servem para medir se a LLM inventa um endpoint.
- Condições de contexto:
    - A: top-5 do TF-IDF (o sistema da AV1);
    - B: top-5 + 5 operações relacionadas pelo grafo;
    - C: top-10 do TF-IDF, que serve de controle (mais contexto sem usar o grafo);
    - D: top-5 + ordem de chamadas.
- Modelos: qwen2.5 com 3B, 7B e 14B de parâmetros, com janela de contexto de 8192 tokens.
- Semente fixa (42) e temperatura 0 na reescrita da consulta e 0,1 na resposta, para reduzir a variação entre execuções. Mesmo assim, a execução na GPU não é totalmente determinística: entre as 3 repetições, o texto das respostas varia (a lista de chamadas citadas mudou em 76 dos 240 grupos modelo × condição × pergunta), mas o acerto mudou em apenas 8 grupos, 5 deles no modelo de 3B. Por isso os resultados são reportados como média das repetições.
- Volume: 3 modelos × 4 condições × 20 perguntas × 3 repetições, num total de 720 respostas.
- Avaliação automática e determinística de cada resposta. Uma chamada é qualquer MÉTODO /path ou curl -X MÉTODO https://api.github.com/... escrito no texto; os segmentos {parâmetro} do corpus aceitam qualquer valor ({owner}, <OWNER>, octocat):
    - Operação certa: uma operação do gabarito aparece em qualquer lugar do texto; nas perguntas fora do corpus, acerta se não citou nenhuma chamada;
    - Endpoint inventado: alguma chamada do texto cujo método e path não existem no corpus (método errado num path que existe também conta);
    - Pré-requisito: nas perguntas que têm pré-requisito e com a operação certa, aparece no texto uma chamada de cada passo do pré-requisito;
    - Uso do contexto: cruza o acerto com a operação certa estar no contexto, em quatro casos: trouxe e usou, respondeu de memória, trouxe e não usou, busca falhou;
    - Tokens de entrada e de saída e tempo de geração.
- Comando: bin/llmExperiment -models qwen2.5:3b,qwen2.5:7b,qwen2.5:14b -reps 3 -out artefatos/experimento_llm. As métricas podem ser recalculadas a partir das respostas gravadas, sem rodar a LLM de novo, com bin/llmExperiment -rescore.
- Saídas: artefatos/experimento_llm.csv, .jsonl e .txt.

12. Resultados e gráficos

12.1 Custo do grafo

A Tabela 1 traz a mediana das 30 execuções de cada cenário (10 consultas × 3 repetições), com o corpus completo (N = 1224) e com a metade (N = 612).

Tabela 1 – Custo de construção e de busca do grafo (mediana)

Medida	**N = 612**	**N = 1224**	**Crescimento**
Vértices / arestas do grafo	1130 / 2443	2214 / 4737	×2,0
Construção do grafo	440 µs	970 µs	×2,2
Busca TF-IDF (top-5)	93 µs	170 µs	×1,8
BFS: arestas examinadas / tempo	74 / 17 µs	102 / 13 µs	×1,4 nas arestas
Dijkstra: arestas examinadas / tempo	216 / 34 µs	356 / 52 µs	×1,6 nas arestas
Plano: arestas examinadas / tempo	5 / 27 µs	5 / 19 µs	constante
Memória do processo	4,5 MB	8,4 MB	o grafo soma cerca de 0,4 MB

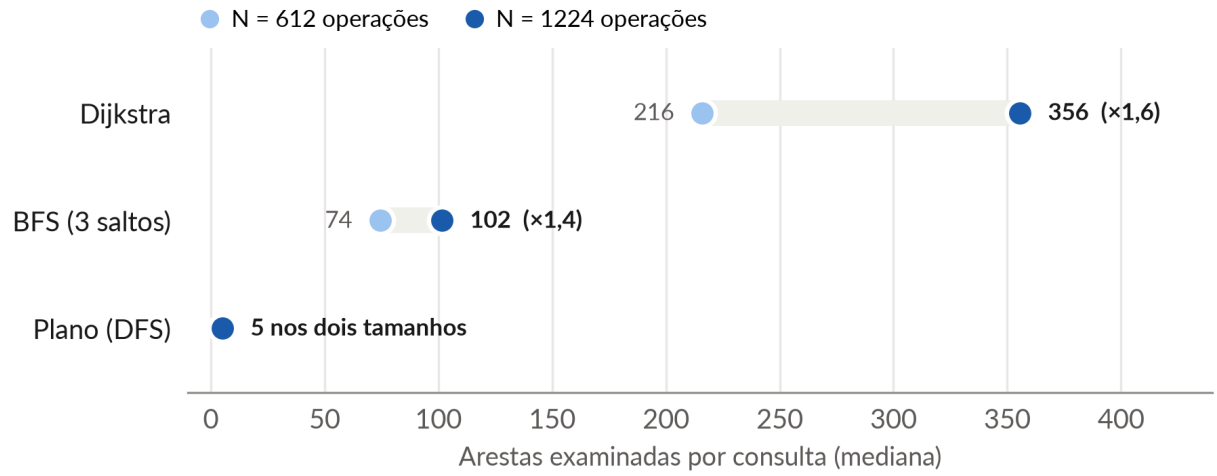


Figura 1 – Arestas examinadas por consulta em cada algoritmo, com 612 e 1224 operações

- O grafo cresce na mesma proporção do corpus: dobrar N dobra V e E, e a construção também fica cerca de duas vezes mais lenta.
- A expansão só percorre a vizinhança das sementes, limitada por H = 3 e B = 3. Por isso o trabalho cresce menos que o corpus: ×1,4 na BFS e ×1,6 no Dijkstra, que examina mais arestas porque relaxa todas as vizinhas de cada vértice fechado.
- O plano só sobe pelas arestas DependsOn até a raiz do path, então examina 5 arestas nos dois tamanhos.

- Todo o custo do grafo fica em microssegundos, bem abaixo do tempo de resposta da LLM, que é de segundos.
- A expansão não muda o acerto da busca: a operação certa aparece no resultado em 12 de 30 execuções com N = 612 e em 27 de 30 com N = 1224, os mesmos números da busca sem grafo. O grafo acrescenta operações vizinhas, mas não recupera uma resposta que o TF-IDF não trouxe.

12.2 Experimento com a LLM

Foram 720 respostas (3 modelos × 4 condições × 20 perguntas × 3 repetições). A Tabela 2 mostra o acerto da operação nas 16 perguntas com resposta no corpus (48 respostas por célula) e o custo de cada condição.

Tabela 2 – Acerto da operação por modelo e condição, e custo de cada condição

Modelo	**A: top-5**	**B: top-5 + relacionadas**	**C: top-10**	**D: top-5 + plano**
qwen2.5 3B	54%	50%	58%	52%
qwen2.5 7B	71%	75%	75%	69%
qwen2.5 14B	81%	75%	81%	81%
Tokens de entrada (média)	2150	2363 (+10%)	3439 (+60%)	2223 (+3%)
Operação certa no contexto	99 de 144	114 de 144	105 de 144	99 de 144

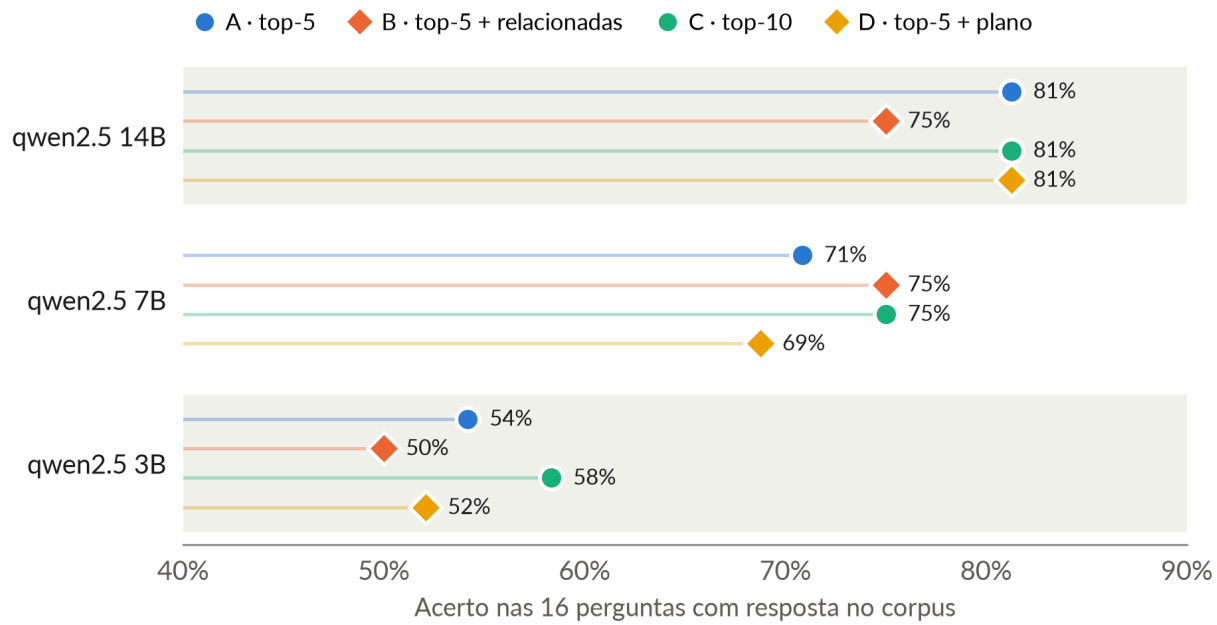


Figura 2 – Acerto da operação por modelo e condição de contexto

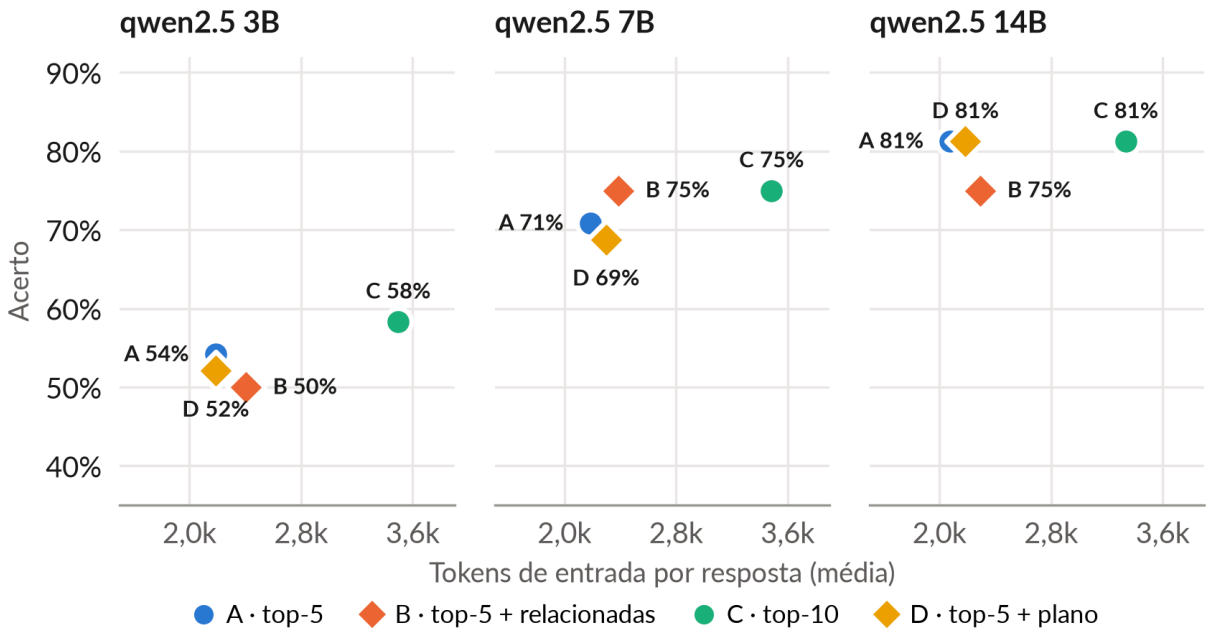


Figura 3 – Tokens de entrada e acerto de cada condição, por modelo

Tabela 3 – Uso do contexto nas perguntas com resposta no corpus (144 respostas por condição)

Caso	A	B	C	D
Trouxe e usou	82	90	85	82
Trouxe e não usou	17	24	20	17
Acertou de memória	17	6	18	15
Busca não trouxe e errou	28	24	21	30

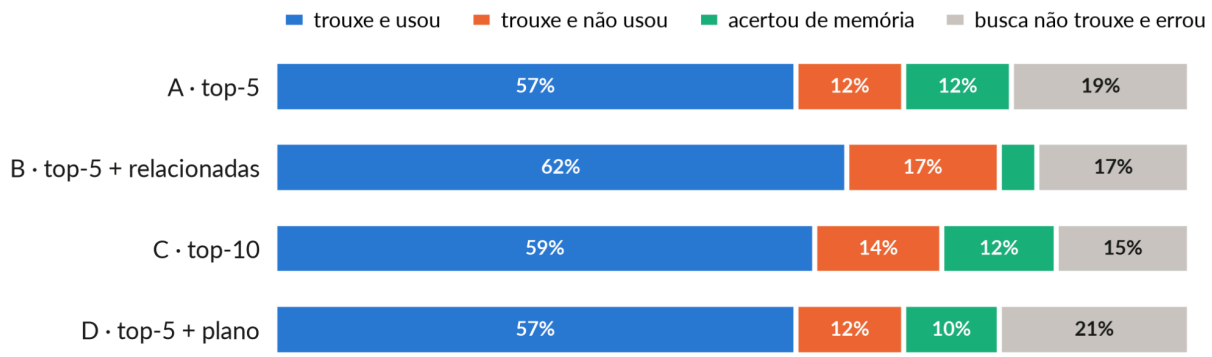


Figura 4 – Uso do contexto em cada condição

Nas perguntas que exigem um passo anterior, por exemplo listar as releases antes de excluir uma delas, a Tabela 4 mostra quantas das respostas certas citaram esse passo.

Tabela 4 – Respostas certas que citaram o pré-requisito

Modelo	**A**	**B**	**C**	**D**
qwen2.5 3B	0 de 10	0 de 9	3 de 12	5 de 9
qwen2.5 7B	0 de 16	3 de 18	0 de 18	0 de 15
qwen2.5 14B	0 de 15	0 de 15	0 de 15	12 de 15
Total	0 de 41 (0%)	3 de 42 (7%)	3 de 45 (7%)	17 de 39 (44%)

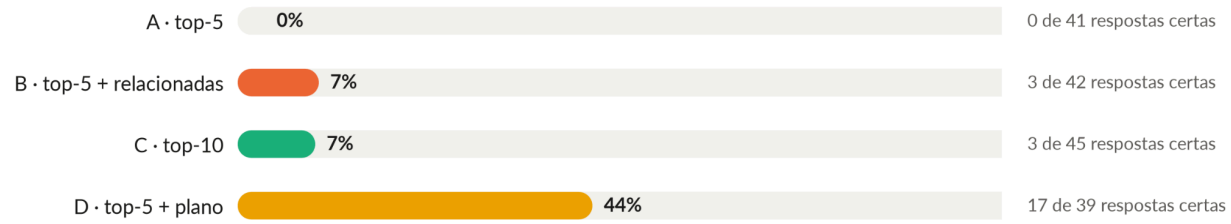


Figura 5 – Respostas certas que citaram o pré-requisito, por condição

Leitura dos resultados

- O tamanho do modelo pesa mais que a condição de contexto: o 3B acerta entre 50% e 58%, o 7B entre 69% e 75% e o 14B entre 75% e 81%.
- Entre as condições, as diferenças são de poucos pontos e mudam de direção conforme o modelo. Com 48 respostas por célula, 4 pontos são 2 respostas, então não dá para afirmar que o grafo melhora o acerto da operação.
- Mais contexto sem grafo (C) custa 60% a mais de tokens e ganha no máximo 4 pontos sobre A.
- As relacionadas (B) põem a operação certa no contexto com mais frequência (114 contra 99 de 144), mas parte desse ganho se perde: a LLM recebe a operação e não a usa em 24 respostas.
- O ganho do grafo aparece na ordem das chamadas: com o plano (D), 44% das respostas certas citam o pré-requisito, contra 0% em A, com só 3% a mais de tokens. O efeito vem principalmente do 14B (12 de 15). O 7B ignora o plano (0 de 15).
- Nas 4 perguntas sem resposta no corpus, a LLM só deixou de citar uma chamada em 10 de 144 respostas, em qualquer condição. O contexto não impede que ela sugira um endpoint. Chamadas inexistentes no corpus aparecem em cerca de 8% das respostas (13 a 17 de 180 por condição), sem diferença clara entre as condições.

12.3 Comparação com o baseline

O baseline é o sistema da AV1: busca TF-IDF (-preset 2) devolvendo o top-k, sem grafo (seção 5.4). No experimento com a LLM, ele corresponde à condição A.

- Busca: a expansão pelo grafo não aumentou o número de consultas em que a operação certa aparece (12 de 30 com N = 612 e 27 de 30 com N = 1224, iguais ao baseline). Ela só acrescenta operações vizinhas às que o TF-IDF já achou.
- Custo: o grafo é construído uma vez ao carregar o corpus (menos de 1 ms) e soma de 13 a 52 µs por consulta na expansão, além de cerca de 0,4 MB de memória.
- Acerto da operação pela LLM: B e D ficam até 6 pontos acima ou abaixo de A, conforme o modelo. Nenhuma condição supera o baseline de forma consistente.
- Pré-requisito: é onde o grafo supera o baseline. Em D, 17 de 39 respostas certas citam o passo anterior, contra 0 de 41 em A, com 3% a mais de tokens.
- Controle sem grafo (C): dar o dobro de operações ao baseline custa 60% a mais de tokens e quase não traz o pré-requisito (3 de 45).

13. Relação com IA generativa e RAG

O sistema funciona como um RAG (geração aumentada por recuperação): a LLM não consulta a documentação sozinha. Ela recebe as operações devolvidas pelo motor de busca e responde com base nelas. O motor de busca é exposto como a tool search_github_api de um servidor MCP, e a ponte com o Ollama segue três passos: reescreve a pergunta em português como uma consulta em inglês, chama a busca sempre (a LLM não pode pular a recuperação) e gera a resposta com as operações no contexto. O grafo entra no passo de recuperação: além do top-k, a tool pode devolver operações relacionadas (condição B) ou a ordem de chamadas até a operação (condição D).

O experimento mostra quatro pontos sobre o uso de RAG neste corpus:

- A recuperação limita a resposta. Quando a busca não traz a operação certa, a LLM erra na maior parte das vezes: entre 21 e 30 de 144 respostas por condição caem nesse caso, contra 6 a 18 que acertam de memória.
- Mais contexto não é melhor contexto. Dobrar o top-k (C) aumentou os tokens em 60% sem ganho claro, e mesmo com a operação certa no contexto a LLM deixou de usá-la em 17 a 24 de 144 respostas.
- Contexto estruturado muda a forma da resposta. O plano de chamadas (D) é curto e diz a ordem dos passos, e foi a única condição em que a LLM passou a explicar o pré-requisito, principalmente no modelo de 14B.
- O RAG não impede a alucinação. Nas perguntas sem resposta no corpus, a LLM citou alguma chamada em 134 de 144 respostas, e cerca de 8% das respostas trazem chamadas que não existem no corpus.

Para quem usa a documentação, isso indica que o ganho de um RAG sobre APIs vem mais de recuperar bem e de entregar o contexto em forma de passos do que de aumentar o volume de texto enviado ao modelo.

Utilização dos algoritmos: Os algoritmos são utilizados para selecionar e organizar as informações coletadas, que serão enviadas para a LLM para a geração das respostas. O BFS e Dijkstra buscam as operações relacionadas à pergunta e o DFS organiza a ordem das operações para identificar a ordem em que as operações precisam ser executadas. Assim, o grafo permite que as dependências das operações sejam identificadas, fornecendo a sequência de execução para a LLM.

Impacto no prompt, tokens e latência: Ocorreu um aumento de 60% no consumo de tokens ao dobrar a quantidade de operações recuperadas na busca inicial, de 5 para 10. Já ao fornecer um plano de chamadas estruturado, foi observado um aumento de apenas 3% no consumo. Isso indica que ao fornecer informações mais organizadas podem ser apresentadas respostas mais completas e coerentes com o contexto fornecido, sem aumentar significativamente o tempo de processamento e o custo para formulação das respostas.

Contexto enviesado e riscos de perder informações relevantes: Se a busca inicial não encontrar as operações corretas, o grafo pode não conseguir recuperar as informações para a formulação da resposta. As operações relacionadas que o grafo encontrar podem não ser úteis para responder a pergunta. O caminho que possui o menor custo no grafo pode não representar as informações mais relevantes para as respostas. A LLM pode ignorar ou inventar informações durante a formulação das respostas, principalmente quando o contexto já está muito grande.

Verificação das afirmações da LLM: Ao utilizar um plano de chamadas estruturado, a LLM passou a apresentar os pré-requisitos das operações. Mas em 8% das respostas, foram utilizadas informações que não existiam na documentação. Dessa forma, é importante verificar as respostas fornecidas e definir algumas regras para evitar esse comportamento, como definir que o modelo utilize apenas as informações presentes na documentação fornecida e informe quando não houver as informações necessárias para a formulação da resposta.

14. Limitações e ameaças à validade

14.1 Validade interna

- A execução na GPU não é totalmente determinística, mesmo com semente fixa e temperatura baixa. Entre as 3 repetições, a lista de chamadas citadas mudou em 76 dos 240 grupos. O acerto mudou em só 8 grupos, e por isso os resultados são a média das repetições.
- Nas perguntas fora do corpus, a regra conta como erro qualquer chamada citada, mesmo quando a LLM a cita para dizer que a operação não existe. Assim, o acerto fora do corpus (10 de 144) é um limite inferior.
- O gabarito de pré-requisitos foi montado a partir do próprio grafo e depois revisado à mão, o que corrigiu 2 das 8 perguntas. Isso pode favorecer a condição D, que usa o mesmo grafo.

14.2 Validade de construto

- A regra de acerto verifica se a operação certa aparece no texto. Ela não verifica se a resposta explica corretamente os parâmetros, o corpo da requisição ou a autenticação.
- A regra DependsOn só cria dependências para parâmetros terminados em _id. Parâmetros como {issue_number} e {pull_number} não geram aresta, e o plano não mostra esses passos.

14.3 Validade externa

- O gabarito tem só 20 perguntas, das quais 8 têm pré-requisito e 4 não têm resposta no corpus. Com 48 respostas por célula, uma diferença de 4 pontos são 2 respostas.
- Foram testados um único corpus (GitHub REST API) e uma única família de modelos (qwen2.5, de 3B a 14B), rodando localmente. Outros corpora e modelos maiores podem se comportar de outro jeito.
- O experimento de custo usa as 10 perguntas da AV1 e só dois tamanhos de corpus (612 e 1224). As taxas de crescimento são estimativas a partir de dois pontos.

15. Declaração de Uso de IA Generativa

Amintas Gomes de Moraes Neto: não utilizou IA generativa.
Flavio Emanuel de Oliveira Santos: não utilizou IA generativa.

Ismael Teles Santos:

Pergunta	**Resposta**
Qual o modelo utilizado?	Claude Opus 5.5 (Claude Code)
Qual a finalidade de cada uso?	Implementação do algoritmo de avaliação automática das respostas da LLM. Geração dos gráficos com base nos resultados.
Quais os prompts relevantes? (até 5)	"Gere gráficos com os resultados do experimento."
Quais as sugestões aproveitadas?	A estrutura da avaliação automática e os testes da regra. O formato dos gráficos.
Quais as sugestões corrigidas ou rejeitadas?	Alucinou ao sugerir pré-requisitos para algumas perguntas, indicando operações que não eram o passo anterior correto. Foi corrigido manualmente no gabarito.
Quais foram os erros identificados?	Em um gráfico, usou um conjunto de perguntas diferente do descrito na legenda. Foi corrigido.

Quais os testes, provas, documentação ou observações utilizadas para verificação?	Testes unitários da regra de avaliação. Conferência dos números com os arquivos de resultados.

João Fernandes Santos Filho:

Pergunta	**Resposta**
Qual o modelo utilizado?	GPT-5.6 Luna
Qual a finalidade de cada uso?	Dúvidas sobre a implementação do código
	Relação com IA generativa (algoritmos, impacto no consumo e na latência, contexto enviesado, perda de informações, verificação das respostas)

Ântones do Nascimento de Lima:

Pergunta	**Resposta**
Qual o modelo utilizado?	GPT-5.6
Qual a finalidade de cada uso?	Brainstorming inicial
	Revisão das recorrências
	Revisão da análise no modelo RAM
	Análise e reformulação de escrita.

16. Conclusão

Esta etapa acrescentou ao motor de busca da AV1 um grafo de operações da GitHub REST API, com arestas da hierarquia de paths e de dependência (DependsOn), e três formas de percorrê-lo: BFS e Dijkstra para buscar operações relacionadas, e DFS para montar a ordem de chamadas até uma operação.

Do lado algorítmico, o grafo cresce linearmente com o corpus e custa microssegundos por consulta. A expansão examina só a vizinhança das sementes, e o plano examina sempre as mesmas 5 arestas, porque só depende da profundidade do path.

Do lado da qualidade, o grafo não aumentou o acerto da busca nem o acerto da operação pela LLM. Esse acerto dependeu mais do tamanho do modelo (de 50% a 58% no 3B, até 75% a 81% no 14B) do que do

contexto recebido. O ganho apareceu na ordem das chamadas: com o plano, 44% das respostas certas explicaram o passo anterior necessário, contra nenhuma no baseline, com só 3% a mais de tokens.

17. Contribuição individual

Amintas Gomes de Moraes Neto

- Submissão do PR #13: Construção do grafo do corpus;
- Elaboração da seção 9;
- Slides sobre melhor, pior e caso médio.

Flavio Emanuel de Oliveira Santos

- Revisão e merge da PR #13
- Submissão da PR #14: implementação de grafo a partir da concebida pela PR #13 e algoritmos de ordenação topológica, BFS, DFS e Dijkstra.
- Módulo de testes (graph_test.go)
- Documentação da funcionalidade de busca com grafos (README.md)
- Confecção da estrutura do relatório da AV2
- Confecção das partes A (Especificação do problema) e B (Algoritmo e corretude) do relatório
- Slides individuais de apresentação

Ismael Teles Santos

- Submissão da PR #15: integração do grafo ao motor de busca, à LLM e à interface;
- Montagem do grafo no engine, com contadores da expansão e dos planos de chamada;
- Regra de dependências: DependsOn criada só para parâmetros _id, subindo pelos ancestrais do path;
- Tratamento de documentos duplicados (mesmo método e path), mantendo o primeiro;
- Testes do grafo no corpus real, incluindo os casos em que a estratégia falha;
- Cenários do grafo no benchmark (base, BFS, Dijkstra e plano);
- Envio das operações relacionadas e da ordem de chamadas para a LLM pela tool do MCP;
- Gabarito da AV2 e experimento com a LLM nas condições A, B, C e D;
- Redesenho da interface.

Ântones do Nascimento de Lima

- Análise no modelo RAM
- Recorrências

João Fernandes Santos Filho

- Relação com a IA generativa.

Gabriel Gomes Attie

- Resultados

18. URL do vídeo

https://youtu.be/B_CCQpkuctk

19. Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Introduction to Algorithms**. 4. ed. Cambridge: MIT Press, 2022.

DIJKSTRA, E. W. A note on two problems in connexion with graphs. **Numerische Mathematik**, v. 1, p. 269–271, 1959.

GITHUB. **GitHub REST API description** (rest-api-description), descrição OpenAPI versão 1.1.4. Disponível em: https://github.com/github/rest-api-description. Acesso em: 24 set. 2026.

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: **Advances in Neural Information Processing Systems 33** (NeurIPS 2020), 2020.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to Information Retrieval**. Cambridge: Cambridge University Press, 2008.

MODEL CONTEXT PROTOCOL. **Model Context Protocol Specification**. Disponível em: https://modelcontextprotocol.io. Acesso em: 27 out. 2026.

OLLAMA. **Ollama**. Disponível em: https://ollama.com. Acesso em: 27 out. 2026.

OPENAPI INITIATIVE. **OpenAPI Specification**, versão 3.0.3. Disponível em: https://spec.openapis.org/oas/v3.0.3. Acesso em: 27 out. 2026.

QWEN TEAM. **Qwen2.5 Technical Report**. arXiv:2412.15115, 2024.